
KAM? PAF! - Équipe 107

PolyQuiz
Plan de projet

Version 1.7

Historique des révisions

Date	Version	Description	Auteur
2024-09-01	1.0	Rédaction de l'introduction et de l'énoncé des travaux (solution proposée et biens livrables du projet)	Anis Ait-Kaci Ali
2024-09-01	1.1	Rédaction de l'énoncé des travaux (hypothèses et contraintes)	Michlove Pierre, Pungtzé-Sy D. Kamdem
2024-09-02	1.2	Commencement de la gestion et suivi de l'avancement	Michlove Pierre
2024-09-03	1.3	Rédaction finale de l'énoncé des travaux (solution proposée) et équipe de développement.	Anis Ait-Kaci Ali
2024-09-16	1.4	Rédaction de la gestion des exigences, des risques, de configuration et du contrôle de la qualité	Pungtzé-Sy D. Kamdem, Farid Bakir
2024-09-20	1.5	Rédaction d'échéancier et l'entente contractuelle	Pungtzé-Sy D. Kamdem, Farid Bakir
2024-09-23	1.6	Rédaction de l'échéancier	Kevin Habchy
2024-09-25	1.7	Révision du document au complet	Kevin Habchy
2024-11-29	1.8	Correction remise finale	Michlove Pierre, Pungtzé-Sy D. Kamdem, Farid Bakir, Anis Ait-Kaci Ali, Ali Elmoussaoui, Kevin Habchy

Table des matières

1. Introduction	4
2. Énoncé des travaux	4
2.1. Solution proposée	4
2.2. Hypothèses et contraintes	5
2.3. Biens livrables du projet	6
3. Gestion et suivi de l'avancement	6
3.1. Gestion des exigences	6
3.2. Contrôle de la qualité	7
3.3. Gestion de risque	7
3.4. Gestion de configuration	9
4. Échéancier du projet	9
5. Équipe de développement	11
6. Entente contractuelle proposée	12

Plan de projet

1. Introduction

Ce document présente la planification de notre projet pour réaliser le jeu-questionnaire. Il a pour objectif de présenter à la PME québécoise la façon dont nous avons planifié notre projet. Au niveau de l'organisation du document, ce dernier sera divisé en cinq sections. La section 2 traite de l'énoncé des travaux à travers la solution proposée, les hypothèses et contraintes ainsi que les biens livrables du projet. Ensuite, la section 3 aborde la gestion et le suivi de l'avancement à travers la gestion des exigences, le contrôle de la qualité, la gestion de risque et la gestion de configuration. Après cela, la section 4 présente l'échéancier du projet. Subséquemment, la section 5 montre l'équipe de développement. Finalement, la section 6 explique l'entente contractuelle proposée.

2. Énoncé des travaux

2.1. Solution proposée

Pour répondre aux besoins de la PME québécoise PolyApps, nous proposons PolyQuiz. Cette solution est en fait une transformation de notre jeu-questionnaire réalisé en format web pour en faire une application multiplateforme et multimodes. Les utilisateurs pourront jouer au jeu avec des appareils étant compatibles sous Windows 11 et Android 13. Afin d'y arriver, il est nécessaire de fournir une solution comportant un client lourd et un client léger fonctionnant respectivement sous forme d'application de bureau et d'application Android. Additionnellement, un serveur est impératif pour supporter plusieurs parties simultanées, maintenir la communication entre les deux clients et permettre à chaque client d'effectuer ses requêtes. Au niveau des fonctionnalités de notre produit, celui-ci intègre les caractéristiques de base de notre précédente application numérique telles que la création de questionnaires, les parties en mode multijoueur et la fonction de clavardage. En matière de contenu, les clients lourd et léger ne sont pas identiques. En effet, dans le cadre du projet, les vues d'administration et de création d'un questionnaire sont uniquement disponibles pour le client lourd. Sur le plan technique, le client lourd est programmé à l'aide d'Angular, d'Electron et du langage TypeScript, tandis que le client léger est développé à l'aide de Flutter et du langage Dart. Quant au serveur, il est réalisé à l'aide de NestJS.

2.2. Hypothèses et contraintes

Ce plan comporte plusieurs hypothèses :

- Le public cible a une bonne familiarité avec les jeux-questionnaires et est habitué à des interfaces utilisateur modernes et intuitives.
- Les utilisateurs disposeront d'une connexion internet stable pour participer à des parties en temps réel.
- Le développement s'appuie sur le code existant de LOG2990, avec une architecture déjà partiellement définie.
- Les utilisateurs disposeront d'un appareil Android ou d'un ordinateur pour participer à des parties en temps réel.
- Les équipes de développement auront une compétence de base en Angular et Flutter.
- Les services infonuagiques utilisés pour héberger le serveur et la base de données sont fiables et offrent une performance suffisante pour supporter les besoins du projet.
- Les utilisateurs ont les compétences nécessaires pour créer et gérer un compte utilisateur sans assistance supplémentaire.
- Les bibliothèques du client léger et du client lourd devront être compatibles pour la réalisation du projet et l'amélioration du code existant.
- Les outils de collaboration seront adoptés par toute l'équipe pour assurer une communication fluide et un suivi efficace du projet. Parmi ces outils, l'équipe adoptera Git pour le contrôle de version, Discord pour la communication.
- Les parties prenantes sont disponibles pour des retours réguliers et participent activement aux validations des jalons.

Contraintes :

- L'application finale devra concorder avec ce qui sera inscrit dans le document SRS.
- La plateforme doit être compatible sur Windows 11 et Android 13.
- L'équipe aura accès à des ordinateurs de développement, une seule tablette Android pour les tests, et un serveur pour héberger l'application.
- Le projet doit être complété en 15 semaines (soit le 29 novembre 2024), avec des jalons importants tels que la livraison d'un prototype de communication du client lourd et léger à la semaine 4 (soit le 27 septembre 2024).
- La charge de travail requise pour livrer le projet dans une telle équipe doit être de 1080 heures-personnes.
- En vue de la charge de travail précédente et d'un salaire établi à 145 \$/h par gestionnaire, un budget de 125 100 \$ est à allouer au développement du projet.
- Le projet doit respecter les normes internes de qualité de l'équipe ainsi que les bonnes pratiques de l'industrie en termes de sécurité, d'accessibilité et de performance.
- Le client lourd et le client léger devraient être capables de communiquer entre eux et doivent être cohérents.

2.3. Biens livrables du projet

Le projet aura deux biens livrables. Le premier bien livrable consistera à remettre la réponse à l'appel d'offres. Il aura pour artefacts cinq documents dont le plan de projet, le SRS, la liste d'exigences, le document d'architecture logicielle et le protocole de communication et pour code deux éléments, soit les deux prototypes de communication client lourd et léger. Ce livrable devra être remis le 27 septembre 2024. Ensuite, le second bien livrable consistera à remettre le produit final. Il aura pour artefacts trois documents dont la mise à jour des artefacts remis précédemment, le plan de tests et les résultats de tests et pour code le code source et l'exécutable du produit incluant client lourd, client léger et serveur. Ce livrable devra être remis le 29 novembre 2024.

3. Gestion et suivi de l'avancement

3.1. Gestion des exigences

Pour gérer tout changement d'exigences, nous devons d'abord identifier les différentes exigences. Il sera ensuite nécessaire de mettre en œuvre les exigences essentielles en priorité, puis de s'occuper des exigences souhaitables. Pour cela, la fonctionnalité « Issues » GitLab sera utilisée afin d'établir les exigences à réaliser par chaque membre de l'équipe. En utilisant cette fonctionnalité, nous pourrions subdiviser les fonctionnalités en tâches, ce que l'on considère comme des tickets. Les tickets devront avoir un titre simple ainsi qu'une description claire et détaillée de la tâche à réaliser. De plus, chaque ticket devra avoir un poids représentant le temps estimé pour réaliser la tâche en heures-personnes ainsi qu'une date d'échéance, pour la personne qui y sera affectée. Pour chaque ticket, il faudra spécifier s'il est essentiel ou souhaitable et les exigences respectives par leur numéro.

Nous ferons d'ailleurs usage des épopées pour regrouper les tickets d'une même catégorie qui s'étalent sur plusieurs niveaux de manière à préciser l'ordre dans lequel certaines tâches devraient être accomplies. Les tickets de même catégorie et de niveaux différents seront liés ensemble, permettant ainsi de déterminer l'ordre dans lequel les compléter. Les tickets de niveaux similaires seront aussi regroupés par jalons de complétion qu'on appellera sprints pour garantir une évolution constante et une charge de travail hebdomadaire saine pour tous les membres de l'équipe. Ces fonctionnalités devront néanmoins être prêtes à tester avant un jalon de tests une semaine avant leur jalon de complétion pour allouer assez de temps à la vérification du code, la validation des fonctionnalités, la correction des bogues et l'ajustement de l'interface. Naturellement, les dates d'échéances des tickets correspondront aux jalons établis dans l'échéancier ci-dessous. Les tâches identiques entre les clients lourd et léger seront séparées afin de permettre de distinguer leur attribution et leur avancement.

Par la suite, les tickets seront placés dans un tableau interactif suivant la méthode Kanban. Tous les tickets se trouveront d'abord dans la section "À faire". Lorsqu'un ticket sera assigné et qu'il aura été commencé, il sera déplacé à la section "En cours" jusqu'à sa complétion. Une fois terminé, le ticket pourra être mis en zone "À tester" pour qu'il soit vérifié, en termes de logique fonctionnelle, et validé, selon les requis, par au moins deux des membres non assignés. Une fois testé, et au besoin corrigé, le ticket pourra finalement être placé dans la section "Terminé".

Dans l'éventualité où une fonctionnalité ne peut être complétée à temps en raison par exemple d'un niveau de difficulté imprévu, les personnes assignées à la tâche organiseront une réunion avec les autres membres le plus tôt possible pour discuter de la situation. Une analyse d'impact sera effectuée pour déterminer les risques de procéder à un changement d'exigence. Cette analyse prendra en compte l'importance de la tâche, quantifiée par le nombre de points dans la liste d'exigence, par rapport au temps nécessaire à son développement et au temps restant avant la remise finale. Une réallocation de personnes responsables de la tâche peut aussi se produire si certains sont plus habilités à la faire. Elle comprendra aussi d'évaluer si des solutions ne peuvent être trouvées ou sont trop difficiles à mettre en place pour respecter les délais, la faisabilité de l'exigence remplaçante, les fonctionnalités affectées par le changement. Si elle se conclut par un changement d'exigence au sein de l'équipe, tous les documents et la planification seront mis à jour dans l'optique de garder des traces claires de l'historique des changements. La nouvelle exigence engendrera un nouveau ticket et ce dernier sera intégré dans la planification selon les mêmes critères qu'au début de cette section. Ainsi, nous nous assurons de procéder de manière systématique et organisée garantissant un développement fluide.

3.2. Contrôle de la qualité

Afin de s'assurer que les livrables du projet respectent un niveau de qualité professionnel, plusieurs méthodes seront mises en place. Tout d'abord, il y aura deux rencontres par semaine afin que chaque duo de l'équipe puisse présenter l'avancement des fonctionnalités qui sont à leur charge. Chacun pourra ainsi énoncer les blocages rencontrés et les succès accomplis lors de la réalisation de ses tâches.

Dans le cas où, un duo rencontre un problème s'étalant sur au moins une rencontre, il pourra alors consulter les autres membres de l'équipe lors de la prochaine réunion et effectuer un remue-méninge pour trouver une solution. Si aucune solution n'est trouvée, les mesures précisées dans la section précédente seront entreprises.

Si le duo estime que la réalisation de sa tâche est un succès, un autre duo de l'équipe vérifiera que les requis relatifs à sa tâche soient tous satisfaits. Les membres de l'équipe testeront ensuite l'application en prenant soin d'examiner les cas limites pour consolider de la robustesse du code et confirmer l'absence de bogues. Pour être plus précis, les membres de l'équipe utiliseront plusieurs stratégies de tests, parmi lesquelles figurent les tests fonctionnels, de performances, d'interface utilisateur, d'intégrité des données, de sécurité et de contrôle d'accès, d'échec/récupération, de configuration et d'installation. Chacun de ces tests aura pour objectif de s'assurer que les deux clients puissent fonctionner adéquatement ensemble, que les diverses exigences soient respectées, que l'application soit suffisamment stable pour être utilisée sur des appareils Windows 11 et Android 13, et que plusieurs requêtes simultanées puissent être réalisées. Par ailleurs, par manque de temps, nous utiliserons comme technique pour chacune de ces stratégies le test en boîte noire. De plus, l'interface et l'expérience utilisateur seront aussi évaluées par les coéquipiers et le client en jugeant de l'efficacité et de convivialité de son utilisation.

À la suite de l'évaluation, une liste qui décrit précisément les points à améliorer servira alors pour former trois types de tickets : l'interface adéquate, la qualité de code et la validité des requis (incluant la présence de bogues). Les tickets seront marqués par priorité selon l'étendue de l'impact sur les autres fonctionnalités de l'application. Une fois les tickets réglés, le duo pourra alors pousser son code sur une branche avant la branche principale, qu'on appellera pré-principale.

Un autre membre que celui qui a travaillé sur la tâche va alors se charger de vérifier que la fusion de branches se passe sans problème. Une fois fusionnés avec la branche pré-principale, de nouveaux tests seront faits pour s'assurer que cette intégration ne nuise pas aux fonctionnalités déjà présentes. Si elle est nuisible, le même mécanisme de documentation des problèmes par ticket sera utilisé et des solutions devront alors être trouvées pour chaque souci. Une fois trouvée, la branche pré-principale sera fusionnée avec la branche principale.

3.3. Gestion de risque

La description des risques suit la convention suivante :

- Ampleur : sur une échelle de 1 à 10, 10 étant le risque le plus élevé. Cette analyse est basée sur la probabilité d'occurrence du risque, ainsi que ses impacts.
- Description : une description textuelle du risque ainsi que les problèmes attendus.
- Impact : échelle définissant la portée du risque
 - C – critique (affecte le projet en entier)
 - E – élevé (affecte les fonctionnalités principales du système)
 - M – moyen (devrait être maîtrisable en appliquant une stratégie d'atténuation adéquate)
 - F – faible (l'acceptation du risque est une stratégie envisageable)
- Facteurs : aspects (**métriques**) du système pouvant être compromis.
- Stratégie de gestion : mesures à prendre afin de gérer le risque.

RSK.1 – Problèmes de performance sous charge élevée

Ampleur	Description	Impact	Facteurs	Stratégie de gestion
8	Le système peut subir des ralentissements ou des pannes lors de pics de trafic, affectant l'expérience utilisateur.	C	Temps de réponse, capacité serveur	Il faut faire des tests de charge et optimiser les requêtes afin qu'il puisse gérer un grand trafic.

RSK.2 – Conflits de code

Ampleur	Description	Impact	Facteurs	Stratégie de gestion
6	Conflits de code fréquents à cause de la gestion et fusion des branches sur GitLab	M	Cohésion du code, Temps de développement	Mise en place de règles précises lors des commits de code, par exemple, faire un commit à chaque deux ou trois fonctions, empêcher que des développeurs travaillent sur le même fichier de code et ne pas dépasser une certaine limite de temps avant de commit.

RSK.3 – Héritage des problèmes

Ampleur	Description	Impact	Facteurs	Stratégie de gestion
7	Héritage des problèmes non résolus de la version du projet 2.	E	Architecture et performance du système	Élaboration d'une analyse et d'une reconception du système suivie d'une refactorisation du code pour rétablir des fondations solides.

RSK.4 - Apprentissage de nouvelles technologies

Ampleur	Description	Impact	Facteurs	Stratégie de gestion
4	Temps d'apprentissage des nouvelles technologies comme Flutter et NestJS qui pourrait être plus long que prévu.	M	Collaboration, Temps de développement	Planifier un processus d'apprentissage régulier et de transfert de connaissances au sein de l'équipe afin que tous puissent développer de manière indépendante.

RSK.5 - Disponibilité des membres

Ampleur	Description	Impact	Facteurs	Stratégie de gestion
5	Horaires de différents membres de l'équipe ne sont pas toujours compatibles.	M	Collaboration, Prise de décision et avancement	Identifier des périodes réservées pour la rencontre de tous les membres de l'équipe dès le début et les adapter continuellement pendant le projet.

3.4. Gestion de configuration

La gestion de changements se fera conformément à la section 3.1, et le suivi des problèmes, bogues et anomalies, comme mentionné dans la section 3.2. Les artefacts du projet (SRS, plan de projet, architecture logicielle, protocole de communication et prototypes) seront nommés de manière descriptive et suivront une convention de nommage uniforme et seront numérotés selon une structure hiérarchique logique, débutant à 1,0 pour la version initiale. Toute révision apportée à un document sera indiquée par un incrément, par exemple, pour les documents : NomDocument_Version.ext (ex. PlanProjet_1.0.docx) ou encore, pour les prototypes : NomModule_Prototype_Version.ext (ex. ClientLourd_Prototype_1.0.exe).

Le code source sera organisé dans des répertoires correspondant aux modules principaux du projet (/desktop, /mobile, /server), et chaque fichier respectera une convention de nommage. Chaque déploiement du projet sera accompagné d'un tag Git représentant le numéro de version (ex. v1.0.0 pour la première livraison). Nous adopterons la stratégie suivante de branches Git pour organiser le développement. Les branches suivantes seront utilisées pour :

- **main** : contenir les versions stables et prêtes à être déployées.
- **premain** : intégrer les fonctionnalités une par une au système complet dans l'idée d'obtenir une prochaine version stable.
- **feat/<fonctionnalité>** : développer de nouvelles fonctionnalités.
- **bug/<bogue>** : corriger des bogues et anomalies.

Chaque modification ou version sera documentée automatiquement à partir des messages de commit, précisant les ajouts, suppressions ou modifications effectuées pour assurer la traçabilité de l'évolution du produit.

4. Échéancier du projet

Le tableau qui suit contient les différents livrables et les jalons du projet. Chaque livrable comprend de principaux lots de travail accompagnés de leurs dates de début et de fin, et de l'effort estimé pour chacun des lots en heures-personnes. Cette estimation inclut le développement et les tests de fonctionnalités. De ce fait, nous étalerons les quatre livrables prévus sur quatre itérations, que l'on nommera « sprint », de deux semaines. À noter que les lots sans précision du client s'apparentent à la fois au lourd et au léger.

Livrable	Date		Lots de travail	Effort estimé
	Début	Fin		
Réponse à l'appel d'offres	9/14/2024	9/27/2024	Réunions de planification	12
			Réunions de suivi	12
			Complétion de la liste d'exigences	20
			Complétion du document de Spécification des Requis du Système (SRS)	24
			Complétion du plan de projet	17
			Complétion du protocole de communication	24
			Complétion du document d'architecture logicielle	24
			Mise en place de GitLab	15
			Prototypes des clients	36
			Débogage des prototypes	20
Sous-total			204	

Livrable	Date		Lots de travail	Effort estimé
	Début	Fin		
Sprint 1	9/28/2024	10/11/2024	Réunions de planification	12
			Réunions de suivi	12
			*Compte Utilisateur Niveau 1: Authentification	24
			*Clavardage Niveau 1: Intégration	24
			*Clavardage Niveau 2: Canaux de discussion	18
			*Clavardage Niveau 3: Historique de clavardage	12
			*Système Amis Niveau 1: Système d'envoi de demandes d'amis	12
			*Vue d'aministration de Jeux - Client lourd uniquement	12
			*Vue de création de Jeux - Client lourd uniquement	18
			*Jeu Niveau 1: Création rapide de partie	30
			*Jeu Niveau 2: Rejoindre une partie	24
			*Jeu Niveau 3: Mode de jeu Classique	36
			Débogage des intégrations	36
Sous-total				270
Sprint 2	10/12/2024	10/25/2024	Réunions de planification	12
			Réunions de suivi	12
			*Avatar	9
			*Personnalisation de l'application	30
			*Compte Utilisateur Niveau 2: Paramètres et statistiques de compte	24
			*Système Amis Niveau 2: Parties "amis seulement"	18
			*Observateurs Niveau 1: Mode Observateur pour les jeux déjà en cours	18
			Complétion du plan de tests logiciels	30
			Débogage des intégrations	36
Sous-total				189
Sprint 3	10/26/2024	11/8/2024	Réunions de planification	12
			Réunions de suivi	12
			*Jeu Niveau 4: Mode de jeu Classé	36
			*Monnaie virtuelle Niveau 1: Gains et dépenses	24
			*Monnaie virtuelle Niveau 2: Partie avec un prix d'entrée	18
			*Système Expérience	18
			*Évaluation Jeux	12
			Débogage des intégrations	36
Sous-total				168

Livrable	Date		Lots de travail	Effort estimé
	Début	Fin		
Remise finale	09/11/2024	22/11/2024	Réunions de planification	12
			Réunions de suivi	12
			*Observateurs Niveau 2: Système de pari	30
			*Classement Joueurs	18
			*Bonhomme pendu	18
			*Mode de visualisation - Client léger uniquement	18
			*Mode vocal - Client léger uniquement	9
			Débogage des intégrations	36
			Peaufinement de l'expérience utilisateur	30
			Remise de la liste d'exigences	3
			Révision du document de Spécification des Requis du Système (SRS)	6
			Révision du plan de projet	6
			Révision du protocole de communication	12
			Révision du document d'architecture logicielle	12
			Révision du plan de tests logiciels	6
			Exécution et consignation des résultats de tests logiciels	18
			Génération de codes sources et exécutables de l'application (lourd et léger)	2
Sous-total			248	
Grand total			1079	
			Nb d'heures à ajouter	1

4.1 Échéancier détaillé du projet

(*) Les tâches dans le tableau et leur niveau ont été redéfinies en tâches plus petites, en fonction de la liste des exigences. Les détails de chacune des tâches sont disponibles sur le Git du projet.

5. Équipe de développement

KEVIN HABCHY

En tant que gestionnaire de projet, il coordonne les efforts de l'équipe pour garantir le respect des délais et la qualité des livrables. Kevin est aussi principalement chargé de l'application au niveau du client lourd. Il a la responsabilité de diriger l'équipe travaillant sur la partie client lourd. Avec une solide connaissance de TypeScript, Angular, et NodeJS NestJS, il excelle en programmation, fort de l'expérience acquise lors de son stage. Ayant développé et testé de nombreuses fonctionnalités durant le projet initial, Kevin est également un testeur aguerri. Par ailleurs, il joue aussi le rôle de designer UX/UI, veillant à créer une interface utilisateur intuitive et attrayante.

ALI ELMOUSSAOUI

Ali a la charge de l'application au niveau du client léger. Grâce à son expérience, il maîtrise bien Dart et Flutter. En plus de ses compétences en frontend, il est également capable de manipuler du code backend pour gérer le serveur avec NestJS. De plus, il participe au déploiement de l'application grâce à ses connaissances en DevOps et en cloud. Ali a aussi la responsabilité de diriger l'équipe travaillant sur la partie client léger, en s'appuyant sur ses compétences

en communication.

FARID BAKIR

Farid est responsable de l'application au niveau du client léger. Grâce à sa connaissance approfondie des tests, il se charge de leur implémentation pour la partie mobile. Fort de l'expérience acquise lors de son stage, il gère l'architecture logicielle de l'application et supervise les choix technologiques liés au projet. De plus, il a pour mission d'assurer la qualité du code en veillant à ce qu'il respecte les bonnes pratiques de développement, tout en collaborant avec l'équipe pour identifier les améliorations possibles et garantir des fonctionnalités performantes.

ANIS AIT-KACI ALI

Anis est responsable de l'application au niveau du client lourd. Grâce à ses expériences passées, il excelle en programmation avec TypeScript ainsi qu'avec les cadriciels Angular et Electron. Il contribue également au développement du serveur en utilisant NestJS. En plus de son rôle axé sur le client lourd, Anis a des compétences en DevOps, ce qui lui permet de participer efficacement à la gestion des déploiements et des infrastructures. En tant que Scrum Master, il facilite les processus agiles au sein de l'équipe, assurant une bonne communication et une organisation optimale pour atteindre les objectifs du projet.

PUNGTZÉ-SY D. KAMDEM

Pungtzé-Sy est responsable de l'application au niveau du client lourd. Il maîtrise Electron, Angular et TypeScript, ce qui lui permet de gérer efficacement le développement du client lourd. Il est également chargé d'implémenter le serveur grâce à ses compétences en NestJS. En tant que designer UX/UI professionnel, il se concentre principalement sur la conception de l'interface utilisateur pour offrir une expérience optimale. Enfin, en tant que chef d'équipe, il supervise le développement et assure une coordination efficace des efforts pour garantir la qualité des livrables.

MICHLOVE PIERRE

Michlove Pierre est responsable du développement au niveau du client lourd. En tant que testeur expérimenté, il s'assure que toutes les fonctionnalités sont rigoureusement vérifiées pour garantir leur bon fonctionnement. En plus de son rôle de testeur, Michlove contribue aux tâches DevOps en participant au déploiement et à la gestion des infrastructures. Il joue également un rôle clé en tant que développeur en assurance qualité, où il veille à ce que le code respecte les standards de qualité et soit optimisé pour la maintenabilité.

6. Entente contractuelle proposée

Voici notre soumission en réponse à l'appel d'offres pour la création de l'application PolyQuiz. Notre équipe se compose de 6 membres, incluant 5 développeurs à 110\$/h et un gestionnaire de projet à 145 \$/h. Le projet sera géré sur une base de 1080 heures-personnes, avec un coût total estimé à 125 100 \$. Nous proposons une entente contractuelle clé en main, garantissant au client un budget maîtrisé, sans risque de dépassement. Ce type de contrat confie la gestion complète du projet à l'entrepreneur, permettant ainsi au client de suivre l'avancement de PolyQuiz sans devoir gérer chaque étape, allégeant ainsi sa charge de travail. La date de remise de la proposition est fixée au 27 septembre 2024, et la livraison finale de l'application est prévue pour le 29 novembre 2024. Grâce à cette approche contractuelle, nous garantissons un projet livré dans les délais et conforme aux exigences de l'appel d'offres.